

23. Deployment Strategies (Chiến lược triển khai) cho Spring Boot

23.1. Giới thiệu về Deployment Strategies

Triển khai (Deployment) là quá trình đưa ứng dụng từ môi trường phát triển sang môi trường sản xuất để người dùng cuối có thể truy cập và sử dụng. Đối với các ứng dụng Spring Boot, có nhiều chiến lược triển khai khác nhau, mỗi chiến lược có những ưu và nhược điểm riêng, phù hợp với các yêu cầu về khả năng mở rộng, tính sẵn sàng, thời gian downtime, và môi trường vận hành.

Việc lựa chọn chiến lược triển khai phù hợp là rất quan trọng để đảm bảo ứng dụng hoạt động ổn định, hiệu quả và có thể được cập nhật liên tục mà không gây gián đoạn lớn cho người dùng.

23.2. Các loại đóng gói ứng dụng Spring Boot

Trước khi đi sâu vào các chiến lược triển khai, cần hiểu cách Spring Boot đóng gói ứng dụng:

23.2.1. Executable JAR (JAR thực thi)

Đây là cách đóng gói mặc định và phổ biến nhất cho các ứng dụng Spring Boot. Một JAR thực thi chứa tất cả các dependency (bao gồm cả máy chủ nhúng như Tomcat, Jetty, hoặc Undertow) và có thể chạy trực tiếp bằng lệnh `java -jar`.

- **Ưu điểm:** Đơn giản, dễ dàng đóng gói và chạy, không cần cài đặt máy chủ ứng dụng riêng.
- **Nhược điểm:** Kích thước file JAR có thể lớn, không phù hợp với các môi trường yêu cầu WAR truyền thống.

23.2.2. WAR (Web Application Archive)

Nếu bạn cần triển khai ứng dụng Spring Boot lên một máy chủ ứng dụng truyền thống (như Apache Tomcat, WildFly, WebLogic) thay vì sử dụng máy chủ nhúng, bạn có thể đóng gói

ứng dụng dưới dạng file WAR.

- **Ưu điểm:** Tương thích với các hạ tầng hiện có sử dụng máy chủ ứng dụng truyền thống, có thể tận dụng các tính năng quản lý của máy chủ ứng dụng.
- **Nhược điểm:** Phức tạp hơn JAR thực thi, yêu cầu cấu hình máy chủ ứng dụng riêng.

Để đóng gói thành WAR, bạn cần:

1. Thay đổi `packaging` trong `pom.xml` thành `war` .
2. Loại trừ dependency của máy chủ nhúng (ví dụ: `spring-boot-starter-tomcat`) với scope `provided` .
3. Kế thừa `SpringBootServletInitializer` trong lớp `main` của ứng dụng.

23.3. Các chiến lược triển khai phổ biến

23.3.1. In-place Deployment (Triển khai tại chỗ)

Đây là chiến lược đơn giản nhất, trong đó phiên bản mới của ứng dụng được triển khai trực tiếp lên cùng một máy chủ, thay thế phiên bản cũ.

- **Cách thực hiện:** Dừng ứng dụng cũ, sao chép file JAR/WAR mới vào, khởi động lại ứng dụng.
- **Ưu điểm:** Đơn giản, dễ thực hiện.
- **Nhược điểm:** Gây ra downtime cho ứng dụng trong quá trình triển khai. Không phù hợp với các ứng dụng yêu cầu tính sẵn sàng cao.

23.3.2. Rolling Deployment (Triển khai cuốn chiếu)

Trong chiến lược này, các instance của ứng dụng được cập nhật từng bước một. Khi một instance được cập nhật và sẵn sàng, lưu lượng truy cập sẽ được chuyển hướng đến nó, sau đó instance tiếp theo được cập nhật. Quá trình này tiếp tục cho đến khi tất cả các instance được cập nhật.

- **Yêu cầu:** Cần có nhiều instance của ứng dụng và một bộ cân bằng tải (Load Balancer).

- **Cách thực hiện:** Cập nhật một nhóm nhỏ các instance, kiểm tra, sau đó tiếp tục với nhóm khác.
- **Ưu điểm:** Giảm thiểu downtime (hoặc không có downtime nếu được cấu hình tốt), cho phép rollback dễ dàng nếu có vấn đề.
- **Nhược điểm:** Phức tạp hơn In-place, có thể có các vấn đề tương thích ngược nếu phiên bản mới và cũ chạy đồng thời.

23.3.3. Blue/Green Deployment (Triển khai Xanh/Xanh lam)

Chiến lược này sử dụng hai môi trường sản xuất giống hệt nhau: một môi trường "Blue" (hiện đang hoạt động) và một môi trường "Green" (mới được triển khai). Phiên bản mới của ứng dụng được triển khai lên môi trường Green. Sau khi kiểm thử kỹ lưỡng trên môi trường Green, lưu lượng truy cập sẽ được chuyển hướng từ Blue sang Green bằng cách thay đổi cấu hình của bộ cân bằng tải hoặc DNS.

- **Ưu điểm:** Không có downtime, rollback tức thì bằng cách chuyển hướng lưu lượng trở lại môi trường Blue.
- **Nhược điểm:** Yêu cầu tài nguyên gấp đôi (hai môi trường sản xuất), phức tạp trong việc quản lý dữ liệu nếu có thay đổi schema.

23.3.4. Canary Deployment (Triển khai chim hoàng yến)

Canary Deployment là một chiến lược triển khai dần dần, trong đó phiên bản mới của ứng dụng được triển khai cho một nhóm nhỏ người dùng (hoặc một phần nhỏ lưu lượng truy cập). Nếu phiên bản mới hoạt động ổn định, nó sẽ dần dần được triển khai cho nhiều người dùng hơn cho đến khi thay thế hoàn toàn phiên bản cũ.

- **Ưu điểm:** Giảm thiểu rủi ro, cho phép phát hiện sớm các vấn đề trong môi trường sản xuất với tác động hạn chế.
- **Nhược điểm:** Phức tạp trong việc quản lý lưu lượng truy cập và giám sát, cần các công cụ hỗ trợ.

23.4. Triển khai Spring Boot với Docker

Docker đã trở thành một công cụ không thể thiếu trong việc triển khai ứng dụng hiện đại, đặc biệt là microservices. Docker cho phép đóng gói ứng dụng và tất cả các dependency của nó vào một container độc lập, có thể chạy nhất quán trên mọi môi trường.

23.4.1. Tạo Dockerfile

Plain Text

```
# Sử dụng một base image của OpenJDK
FROM openjdk:17-jdk-slim

# Đặt biến môi trường cho Spring Boot JAR
ARG JAR_FILE=target/*.jar

# Sao chép JAR file vào container
COPY ${JAR_FILE} app.jar

#Expose cổng mà ứng dụng Spring Boot lắng nghe
EXPOSE 8080

# Lệnh để chạy ứng dụng khi container khởi động
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

23.4.2. Build Docker Image

Bash

```
docker build -t my-spring-app:1.0 .
```

23.4.3. Run Docker Container

Bash

```
docker run -p 8080:8080 my-spring-app:1.0
```

23.4.4. Ưu điểm của Docker cho Spring Boot

- **Tính nhất quán:** Ứng dụng chạy giống nhau trên mọi môi trường (dev, test, prod).
- **Cô lập:** Các ứng dụng được cô lập với nhau và với hệ thống host.

- **Khả năng mở rộng:** Dễ dàng mở rộng bằng cách chạy nhiều container.
- **Triển khai nhanh chóng:** Container khởi động nhanh hơn máy ảo.

23.5. Triển khai Spring Boot lên Cloud Platforms

Các nền tảng đám mây (Cloud Platforms) như AWS, Google Cloud Platform (GCP), Azure, Heroku, và Pivotal Cloud Foundry (PCF) cung cấp các dịch vụ quản lý để triển khai và vận hành ứng dụng Spring Boot một cách dễ dàng và hiệu quả.

23.5.1. Triển khai lên Heroku (Ví dụ đơn giản)

Heroku là một nền tảng PaaS (Platform as a Service) phổ biến, rất dễ sử dụng để triển khai các ứng dụng Spring Boot.

1. **Cấu hình Procfile** : Tạo file `Procfile` trong thư mục gốc của dự án:
2. **Đẩy code lên Heroku Git:**

Heroku sẽ tự động phát hiện ứng dụng Spring Boot, build nó và triển khai.

23.5.2. Triển khai lên Kubernetes

Kubernetes là một nền tảng điều phối container (container orchestration) mạnh mẽ, lý tưởng để quản lý các ứng dụng microservices được đóng gói trong Docker. Triển khai Spring Boot lên Kubernetes bao gồm:

1. **Đóng gói ứng dụng thành Docker Image.**
2. **Viết Kubernetes Deployment YAML:** Định nghĩa số lượng replicas, image, cổng, biến môi trường, v.v.
3. **Viết Kubernetes Service YAML:** Định nghĩa cách truy cập ứng dụng từ bên ngoài (LoadBalancer, NodePort, ClusterIP).
4. **Triển khai YAML lên Kubernetes Cluster.**

Ưu điểm của Cloud Platforms và Kubernetes:

- **Khả năng mở rộng tự động:** Tự động điều chỉnh tài nguyên theo nhu cầu.
- **Tính sẵn sàng cao:** Đảm bảo ứng dụng luôn hoạt động ngay cả khi có lỗi.
- **Quản lý dễ dàng:** Giảm gánh nặng vận hành.
- **Tích hợp CI/CD:** Dễ dàng tích hợp vào quy trình CI/CD.

23.6. Bản chất của Deployment Strategies

Bản chất của Deployment Strategies là quản lý rủi ro và tối ưu hóa tính sẵn sàng trong quá trình cập nhật ứng dụng. Mỗi chiến lược đều nhằm mục đích giảm thiểu hoặc loại bỏ downtime, đảm bảo trải nghiệm người dùng liền mạch và cung cấp khả năng rollback nhanh chóng khi có vấn đề. Từ việc đóng gói ứng dụng thành JAR/WAR, đến việc sử dụng Docker để tạo môi trường nhất quán, và cuối cùng là triển khai lên các nền tảng đám mây hoặc Kubernetes để tận dụng khả năng tự động hóa và mở rộng, tất cả đều hướng tới mục tiêu xây dựng một quy trình triển khai hiệu quả, đáng tin cậy và tự động. Việc lựa chọn chiến lược phù hợp phụ thuộc vào yêu cầu cụ thể của dự án, mức độ chấp nhận rủi ro và nguồn lực sẵn có.